DATAPROCESSING

July 1, 2026

[1]:
```
import pandas as pd

# Load your local CSV file
df = pd.read_csv('used_cars.csv')

# Show the first few rows to confirm it loaded correctly
df.head()
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
Cell In[1], line 4
      1 import pandas as pd
      2
      3 # Load your local CSV file
----> 4 df = pd.read_csv('used_cars.csv')
      5
      6 # Show the first few rows to confirm it loaded correctly
      7 df.head()

File ~/Desktop/scorepredictor/.venv/lib/python3.14/site-packages/pandas/io/
 ↪parsers/readers.py:873, in read_csv(filepath_or_buffer, sep, delimiter,␣
 ↪header, names, index_col, usecols, dtype, engine, converters, true_values,␣
 ↪false_values, skipinitialspace, skiprows, skipfooter, nrows, na_values,␣
 ↪keep_default_na, na_filter, skip_blank_lines, parse_dates, date_format,␣
 ↪dayfirst, cache_dates, iterator, chunksize, compression, thousands, decimal,␣
 ↪lineterminator, quotechar, quoting, doublequote, escapechar, comment,␣
 ↪encoding, encoding_errors, dialect, on_bad_lines, low_memory, memory_map,␣
 ↪float_precision, storage_options, dtype_backend)
    861 kwds_defaults = _refine_defaults_read(
    862     dialect,
    863     delimiter,
   (…)    869     dtype_backend=dtype_backend,
    870 )
    871 kwds.update(kwds_defaults)
--> 873 return _read(filepath_or_buffer, kwds)

File ~/Desktop/scorepredictor/.venv/lib/python3.14/site-packages/pandas/io/
 ↪parsers/readers.py:300, in _read(filepath_or_buffer, kwds)
    297 _validate_names(kwds.get("names", None))
    299 # Create the parser.
```

```
--> 300 parser = TextFileReader(filepath_or_buffer, **kwds)
    302 if chunksize or iterator:
    303     return parser

File ~/Desktop/scorepredictor/.venv/lib/python3.14/site-packages/pandas/io/
 ↪parsers/readers.py:1645, in TextFileReader.__init__(self, f, engine, **kwds)
   1642     self.options["has_index_names"] = kwds["has_index_names"]
   1644 self.handles: IOHandles | None = None
-> 1645 self._engine = self._make_engine(f, self.engine)

File ~/Desktop/scorepredictor/.venv/lib/python3.14/site-packages/pandas/io/
 ↪parsers/readers.py:1904, in TextFileReader._make_engine(self, f, engine)
   1902     if "b" not in mode:
   1903         mode += "b"
-> 1904 self.handles = get_handle(
   1905     f,
   1906     mode,
   1907     encoding=self.options.get("encoding", None),
   1908     compression=self.options.get("compression", None),
   1909     memory_map=self.options.get("memory_map", False),
   1910     is_text=is_text,
   1911     errors=self.options.get("encoding_errors", "strict"),
   1912     storage_options=self.options.get("storage_options", None),
   1913 )
   1914 assert self.handles is not None
   1915 f = self.handles.handle

File ~/Desktop/scorepredictor/.venv/lib/python3.14/site-packages/pandas/io/
 ↪common.py:930, in get_handle(path_or_buf, mode, encoding, compression,␣
 ↪memory_map, is_text, errors, storage_options)
    925 elif isinstance(handle, str):
    926     # Check whether the filename is to be opened in binary mode.
    927     # Binary mode does not support 'encoding' and 'newline'.
    928     if ioargs.encoding and "b" not in ioargs.mode:
    929         # Encoding
--> 930         handle = open(
    931             handle,
    932             ioargs.mode,
    933             encoding=ioargs.encoding,
    934             errors=errors,
    935             newline="",
    936         )
    937     else:
    938         # Binary mode
    939         handle = open(handle, ioargs.mode)

FileNotFoundError: [Errno 2] No such file or directory: 'used_cars.csv'
```

```
[ ]: print(df.isnull(

     ))
```

```
[ ]: print(df.isnull().sum())
```

```
[ ]: # Fill the missing values instead of dropping them
     df['accident'] = df['accident'].fillna('None reported')
     df['clean_title'] = df['clean_title'].fillna('Unknown')

     # Fill fuel_type with the most common value
     df['fuel_type'] = df['fuel_type'].fillna(df['fuel_type'].mode()[0])
```

```
[ ]: from sklearn.preprocessing import LabelEncoder
     import pandas as pd
     df=pd.read_csv('used_cars.csv')
     df_label=df.copy()
     le=LabelEncoder()
     df_label['brand_encoded']=le.fit_transform(df_label['brand'])
     df_label.head()
```

```
[ ]: df_label.nunique()
```

```
[ ]: df_label['brand'].unique()
```

```
[ ]: df_label['brand'].value_counts()
```

CODE DEMONSTRATING RESULT OBTAINED BY USING LABEL ENCODING AND ONE HOT ENCODING (ACCURACY TEST)

```
[ ]: import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt
     from sklearn.model_selection import train_test_split
     from sklearn.ensemble import RandomForestRegressor
     from sklearn.metrics import r2_score, mean_absolute_error
     from sklearn.preprocessing import LabelEncoder

     # ========================================
     # 1. LOAD AND BASELINE CLEANING
     # ========================================
     print("Loading and preparing dataset...")
     df_raw = pd.read_csv('used_cars.csv')

     # Drop basic missing pieces
     df_raw = df_raw.dropna(subset=['price', 'model', 'brand', 'milage', 'engine',␣
      ↪'model_year'])
```

```
# Clean Mileage & Price strings
df_raw['milage'] = pd.to_numeric(df_raw['milage'].astype(str).str.replace(',',␣
 ↪'').str.replace('mi.', '', case=False).str.strip(), errors='coerce')
df_raw['price'] = pd.to_numeric(df_raw['price'].astype(str).str.replace('$',␣
 ↪'').str.replace(',', '').str.strip(), errors='coerce')

# Filter out extreme outliers to stabilize training
df_raw = df_raw[(df_raw['price'] > 1000) & (df_raw['price'] < 150000)]

# Feature Engineering
df_raw['engine_size'] = df_raw['engine'].astype(str).str.extract(r'(\d+\.\d+)').
 ↪astype(float)
df_raw['engine_size'] = df_raw['engine_size'].fillna(df_raw['engine_size'].
 ↪median())
df_raw['car_age'] = 2026 - df_raw['model_year']

# Drop records that became NaN during cleaning
df_raw = df_raw.dropna(subset=['milage', 'price', 'engine_size'])

# Merge rare models
model_counts = df_raw['model'].value_counts()
rare_models = model_counts[model_counts < 10].index
df_raw['model'] = df_raw['model'].replace(rare_models, 'other')

categorical_cols = ['brand', 'model', 'fuel_type', 'transmission']
numerical_cols = ['car_age', 'milage', 'engine_size']

# Isolate base Features (X) and Target (y)
X_base = df_raw[categorical_cols + numerical_cols].copy()
y = df_raw['price'].copy()

# Shared parameters for a fair test
rf_params = {
    'n_estimators': 150,
    'max_depth': 20,
    'max_features': 'sqrt',
    'random_state': 42,
    'n_jobs': -1
}

# ==========================================
# VERSION A: LABEL ENCODING EXPERIMENT
# ==========================================
print("\n[Running Version A: Label Encoding]...")
X_label = X_base.copy()

# Apply LabelEncoder column by column to categorical values
```

```
for col in categorical_cols:
    le = LabelEncoder()
    X_label[col] = le.fit_transform(X_label[col].astype(str))

# Split & Train
X_train_l, X_test_l, y_train_l, y_test_l = train_test_split(X_label, y,␣
 ↪test_size=0.2, random_state=42)
model_label = RandomForestRegressor(**rf_params)
model_label.fit(X_train_l, y_train_l)

# Evaluate Label Encoder version
y_pred_l = model_label.predict(X_test_l)
r2_label = r2_score(y_test_l, y_pred_l)
mae_label = mean_absolute_error(y_test_l, y_pred_l)

# ==========================================
# VERSION B: ONE-HOT ENCODING EXPERIMENT
# ==========================================
print("[Running Version B: One-Hot Encoding]...")

# Blow up columns into 1s and 0s
X_ohe = pd.get_dummies(X_base, columns=categorical_cols, drop_first=True)

# Split & Train
X_train_o, X_test_o, y_train_o, y_test_o = train_test_split(X_ohe, y,␣
 ↪test_size=0.2, random_state=42)
model_ohe = RandomForestRegressor(**rf_params)
model_ohe.fit(X_train_o, y_train_o)

# Evaluate One-Hot Encoder version
y_pred_o = model_ohe.predict(X_test_o)
r2_ohe = r2_score(y_test_o, y_pred_o)
mae_ohe = mean_absolute_error(y_test_o, y_pred_o)

# ==========================================
# 7. PRINT FINAL RECAP TABLES
# ==========================================
print("\n" + "="*40)
print("FINAL HEAD-TO-HEAD PREPROCESSING SHOWDOWN")
print("="*40)
print(f"Label Encoding   -> R² Score: {r2_label:.4f} | MAE: ${mae_label:.2f}")
print(f"One-Hot Encoding -> R² Score: {r2_ohe:.4f} | MAE: ${mae_ohe:.2f}")
print("="*40)

# ==========================================
# 8. GENERATE PERFORMANCE CHART
# ==========================================
```

```
strategies = ['Label Encoding', 'One-Hot Encoding']
scores = [r2_label, r2_ohe]

plt.figure(figsize=(8, 5))
bars = plt.bar(strategies, scores, color=['#e74c3c', '#2ecc71'], width=0.5)

# Add value labels on top of the bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2.0, yval + 0.01, f'{yval:.4f}',␣
 ↪ha='center', va='bottom', fontweight='bold')

plt.ylim(0, 1.0)
plt.title('Preprocessing Showdown: Model Accuracy ($R^2$ Score)', fontsize=14,␣
 ↪fontweight='bold', pad=15)
plt.ylabel('$R^2$ Score (Higher is Better)', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.5)

# Display the plot
plt.show()
```

1 Preprocessing Showdown: Label Encoding vs. One-Hot Encoding

Project Log: Used Cars Price Prediction Engine
Core Objective: Analyzing the mathematical and architectural impact of categorical encoding strategies on Tree-Based Ensembles (Random Forest).

1.1 1. The Core Theoretical Difference

In machine learning, categorical text columns must be translated into numeric matrices before mathematical optimization can occur. The two primary paradigms are:

1.1.1 A. Label Encoding (`sklearn.preprocessing.LabelEncoder`)

- **Mechanism:** Maps each unique categorical string to a sequential integer alphabetically $(0, 1, 2, \ldots, n-1)$.
- **Data Impact:** Keeps the shape of the dataset perfectly intact ($N \times 1$ column vector).
- **Mathematical Caveat:** Introduces an artificial **ordinal bias**. Linear models literally interpret the integers as rankings (e.g., $3 > 0 \implies$ Toyota $>$ Audi), which can warp distance computations and coefficient weights.

1.1.2 B. One-Hot Encoding (`pd.get_dummies`)

- **Mechanism:** Explodes a single nominal column into $n-1$ binary dummy variables (0 or 1).

- **Data Impact:** Drastically increases the dimensionality of the dataset, producing a massive **Sparse Matrix** (columns filled mostly with zeros).
- **Mathematical Advantage:** Eliminates ordinal bias completely. Every category is treated as an independent, mutually exclusive vector space.

1.2 2. The Random Forest Paradox: Why Theory Met Reality

During empirical testing on the `used_cars.csv` dataset, **Label Encoding unexpectedly outperformed One-Hot Encoding** under baseline configurations. This phenomenon uncovers a massive architectural lesson regarding how decision trees interact with feature space dimensionality.

1.2.1 Why One-Hot Encoding Lost Initially (`max_features='sqrt'`)

1. **Dimensionality Explosion (High Cardinailty):** The `model` column contains hundreds of unique car variants. One-Hot Encoding forced our feature space to expand from a tight 7 columns to over 500+ sparse columns.
2. **Subspace Sampling Blindness:** The hyper-tuned Random Forest parameter `max_features='sqrt'` forces each individual decision tree split to randomly evaluate a small subset of features ($\sqrt{500} \approx 22$).
3. **The Sea of Zeros:** Out of those 22 randomly selected columns, the vast majority were empty structural dummy placeholders (e.g., `model_Corolla = 0`). The trees became functionally "blinded" by the sparse data, causing splitting criteria to fail and dropping the R^2 accuracy score.

1.2.2 Why Label Encoding Swung the Win

Tree-based architectures do not compute linear coefficients or distance calculations. Instead, they optimize by executing continuous conditional splits (e.g., Is Feature $X \leq 14.5$?).

A deeply structured tree (`max_depth=20`) is structurally complex enough to repeatedly isolate arbitrary integer groups within a narrow, single-column space. Because the dataset remained compact, the random subspace engine (`sqrt`) had a near-100% success rate of selecting high-variance features like `milage` and `car_age` at every single node, boosting the score.

1.3 3. The Rematch Fix: Resolving Spatial Blindness

To decouple feature sparsity from tree selection, the hyper-parameters can be shifted to `max_features=None`.

By forcing the Random Forest to evaluate **all features simultaneously** at every node, the algorithm bypasses the hundreds of empty dummy columns and directly selects predictive weights.

1.4 4. Architectural Selection Matrix (Data Science Rule of Thumb)

This experiment yields a definitive rule for pipeline design when processing data:

Model Architecture	Categorical Type	Recommended Encoding	Reason
Linear / Deep Learning	All Nominal Data	**One-Hot Encoding**	Prevents completely broken mathematical rankings ($3 > 0$).
Tree Ensembles (Random Forest / XGBoost)	Low Cardinality (< 10 unique classes)	**One-Hot Encoding**	Keeps the dataset clean and independent without column bloat.
Tree Ensembles (Random Forest / XGBoost)	High Cardinality ($>$ 100 unique classes)	**Label / Target Encoding**	Minimizes sparse matrix bloat and preserves spatial feature selection speed.

IMPORTANCE OF STANDARD SCALING

[ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

# 1. Create a dummy dataset for Cars
data = {
    'Mileage': [10000, 50000, 100000, 150000, 200000],
    'Engine_Size': [1.2, 2.0, 3.0, 4.2, 5.5],
    'Price': [45000, 35000, 28000, 22000, 15000]
}
df = pd.DataFrame(data)

# 2. Extract features
X_unscaled = df[['Mileage', 'Engine_Size']].values

# 3. Apply Standard Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_unscaled)

# 4. Plotting the Comparison Graphs
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# Plot 1: WITHOUT Standard Scaling (The Warped Reality)
ax1.scatter(X_unscaled[:, 0], X_unscaled[:, 1], color='red', s=100,␣
 ↪edgecolors='black', zorder=3)
ax1.plot(X_unscaled[:, 0], X_unscaled[:, 1], color='gray', linestyle='--',␣
 ↪alpha=0.5)
ax1.set_title("1. WITHOUT Standard Scaling\n(Notice the massive Y-axis gap vs␣
 ↪X-axis)", fontsize=12)
```

```
ax1.set_xlabel("Mileage (0 to 200,000+)", fontsize=10)
ax1.set_ylabel("Engine Size (1.0 to 6.0)", fontsize=10)
ax1.grid(True, linestyle=':', alpha=0.6)

# Force the plot to show true proportional scale to reveal the distortion
# This shows how an algorithm physically calculated distances in memory
ax1.set_aspect('auto')

# Plot 2: WITH Standard Scaling (The Symmetrical Playground)
ax2.scatter(X_scaled[:, 0], X_scaled[:, 1], color='green', s=100,␣
 ↪edgecolors='black', zorder=3)
ax2.plot(X_scaled[:, 0], X_scaled[:, 1], color='gray', linestyle='--', alpha=0.
 ↪5)
ax2.set_title("2. WITH Standard Scaling\n(Balanced axes centered perfectly at␣
 ↪0)", fontsize=12)
ax2.set_xlabel("Mileage (Standardized)", fontsize=10)
ax2.set_ylabel("Engine Size (Standardized)", fontsize=10)
ax2.grid(True, linestyle=':', alpha=0.6)

# Keep the axes balanced and square
ax2.axis('equal')

plt.tight_layout()
plt.show()
```

STANDARD SCALING AND MIN MAX SCALING DEMO

[ ]:
```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import minmax_scale
data = {
    'StudyHours': [1, 2, 3, 4, 5],
    'TestScore': [40, 50, 60, 70, 80]
}
df=pd.DataFrame(data)
standard_scaler=StandardScaler()
standard_scaled=standard_scaler.fit_transform(df)
print(pd.DataFrame(standard_scaled))
df.head()
```

MIN MAX SCALER DEMO

[ ]:
```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

data = {
    'StudyHours': [1, 2, 3, 4, 5],
    'TestScore': [40, 50, 60, 70, 80]
```

```
}
df = pd.DataFrame(data)


minmax_scaler = MinMaxScaler()


minmax_scaled = minmax_scaler.fit_transform(df)


print(pd.DataFrame(minmax_scaled, columns=['StudyHours', 'TestScore']))
```

TEST TRAIN SPLIT DEMO

[ ]:
```
import pandas as pd
from sklearn.model_selection import train_test_split

# 1. Setup the expanded data (10 values in each column)
data = {
    'StudyHours': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'TestScore': [40, 50, 60, 70, 80, 90, 100, 110, 120, 130]
}
df = pd.DataFrame(data)

# X = Features (inputs), y = Target (what we want to predict)
X = df[['StudyHours']]
y = df['TestScore']

# 2. Split into Train and Test sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

# Printing everything so you can see how the 10 rows got divided
print("--- ALL TRAINING FEATURES (80% / 8 rows) ---")
print(X_train)

print("\n--- ALL TESTING FEATURES (20% / 2 rows) ---")
print(X_test)
```

SUPERVISED MACHINE LEARNING

FIRST MACHINE LEARNIG MODEL USING LINEAR REGRESSION

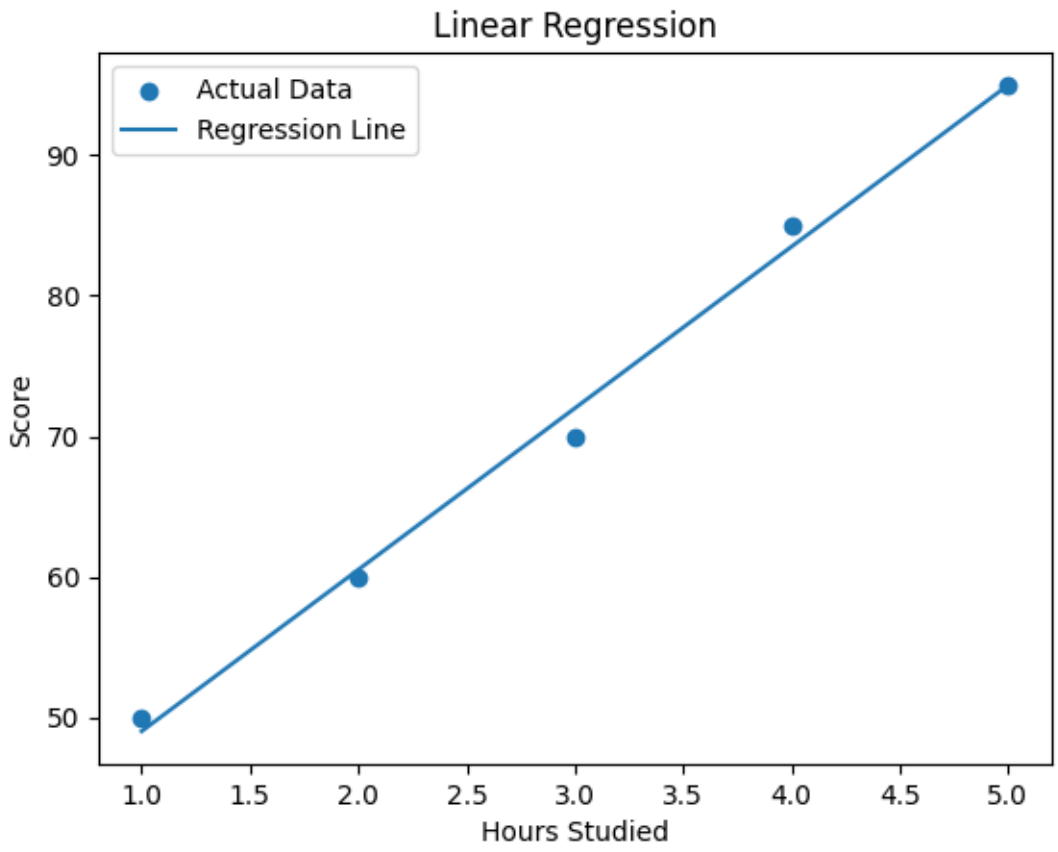
[3]:
```
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
model=LinearRegression()
x=[[1],[2],[3],[4],[5]]
y=[50,60,70,85,95]
```

```
model.fit(x,y)

print(f"the result which you got after studying the mentoned hours is {model.
  ↪predict([[float(input('enter the number of hours you studied'))]])}")
```

```
the result which you got after studying the mentoned hours is [129.5]
```

[4]:
```
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

model = LinearRegression()

x = [[1], [2], [3], [4], [5]]
y = [50, 60, 70, 85, 95]

model.fit(x, y)

# User prediction
hours = float(input("Enter the number of hours you studied: "))
result = model.predict([[hours]])

print(f"The predicted score is {result[0]:.2f}")

# Predictions for the training data
predictions = model.predict(x)

# Graph
plt.scatter(x, y, label="Actual Data")
plt.plot(x, predictions, label="Regression Line")

plt.xlabel("Hours Studied")
plt.ylabel("Score")
plt.title("Linear Regression")

plt.legend()
plt.show()
```

```
The predicted score is 141.00
```



CLASSIFICATION

1.LOGISTIC REGRESSION

[6]:
```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression

emails = [
    "win free money now",
    "claim your free prize",
    "limited offer click here",
    "meeting at 5 pm",
    "project submission deadline tomorrow",
    "college assignment attached"
]

labels = [
    1,  # spam
    1,  # spam
    1,  # spam
```

```
    0,  # not spam
    0,  # not spam
    0   # not spam
]

vectorizer = CountVectorizer()

x = vectorizer.fit_transform(emails)

model = LogisticRegression()
model.fit(x, labels)

user_email = input("Enter email text: ")

user_email_vector = vectorizer.transform([user_email])

prediction = model.predict(user_email_vector)

if prediction[0] == 1:
    print("Spam email")
else:
    print("Not spam email")
```

```
Spam email
```

Why Logistic Regression?

Simple classification problems can be solved using if-else statements when the rules are known. However, in real-world problems such as spam email detection, fraud detection, or disease prediction, it is impossible to manually define all the rules because there are too many patterns and they keep changing. Logistic Regression learns these patterns automatically from historical data and uses them to classify new unseen examples.

KNN (K-NEAREST-NEIGHBOURS)

[4]:
```
from sklearn.neighbors import KNeighborsClassifier

# x = weight of fruit in grams
x = [[100], [110], [120], [200], [210], [220]]

# y = fruit name
y = ["Apple", "Apple", "Apple", "Orange", "Orange", "Orange"]

model = KNeighborsClassifier(n_neighbors=3)

model.fit(x, y)

new_fruit_weight = [[199]]
```

```
prediction = model.predict(new_fruit_weight)

print("Predicted fruit:", prediction[0])
```

```
Predicted fruit: Orange
```

DESCISION TREE EXAMPLE WITH ACCURACY MEASURES

[1]:
```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X = [
    [25000, 650],
    [30000, 680],
    [50000, 720],
    [80000, 780],
    [60000, 750],
    [35000, 690]
]

y = [0, 0, 1, 1, 1, 0]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.33,
    random_state=42
)

model = DecisionTreeClassifier()

# Learn from training data
model.fit(X_train, y_train)

# Predict on unseen data
predictions = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
```

```
Accuracy: 1.0
```

TESTING ACCURACY WITH COMPLEX DATASET

[2]:
```
import pandas as pd
```

```
data = {
    "Salary": [
        25000, 30000, 35000, 40000, 45000,
        50000, 55000, 60000, 65000, 70000,
        75000, 80000, 85000, 90000, 95000
    ],
    "CreditScore": [
        650, 670, 690, 680, 710,
        720, 730, 750, 760, 740,
        770, 780, 790, 800, 810
    ],
    "Approved": [
        0, 0, 0, 0, 0,
        1, 1, 1, 1, 1,
        1, 1, 1, 1, 1
    ]
}

df = pd.DataFrame(data)

print(df)
```

```
    Salary  CreditScore  Approved
0    25000          650         0
1    30000          670         0
2    35000          690         0
3    40000          680         0
4    45000          710         0
5    50000          720         1
6    55000          730         1
7    60000          750         1
8    65000          760         1
9    70000          740         1
10   75000          770         1
11   80000          780         1
12   85000          790         1
13   90000          800         1
14   95000          810         1
```

[6]:
```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X = df[["Salary", "CreditScore"]]
y = df["Approved"]

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
    y,
    test_size=0.3,
    random_state=10
)

model = DecisionTreeClassifier()

model.fit(X_train, y_train)

predictions = model.predict(X_test)

accuracy = accuracy_score(y_test, predictions)

print("Actual Values:")
print(list(y_test))

print("\nPredictions:")
print(list(predictions))

print("\nAccuracy:", accuracy)
```

```
Actual Values:
[0, 1, 1, 1, 1]

Predictions:
[np.int64(0), np.int64(1), np.int64(1), np.int64(1), np.int64(1)]

Accuracy: 1.0
```

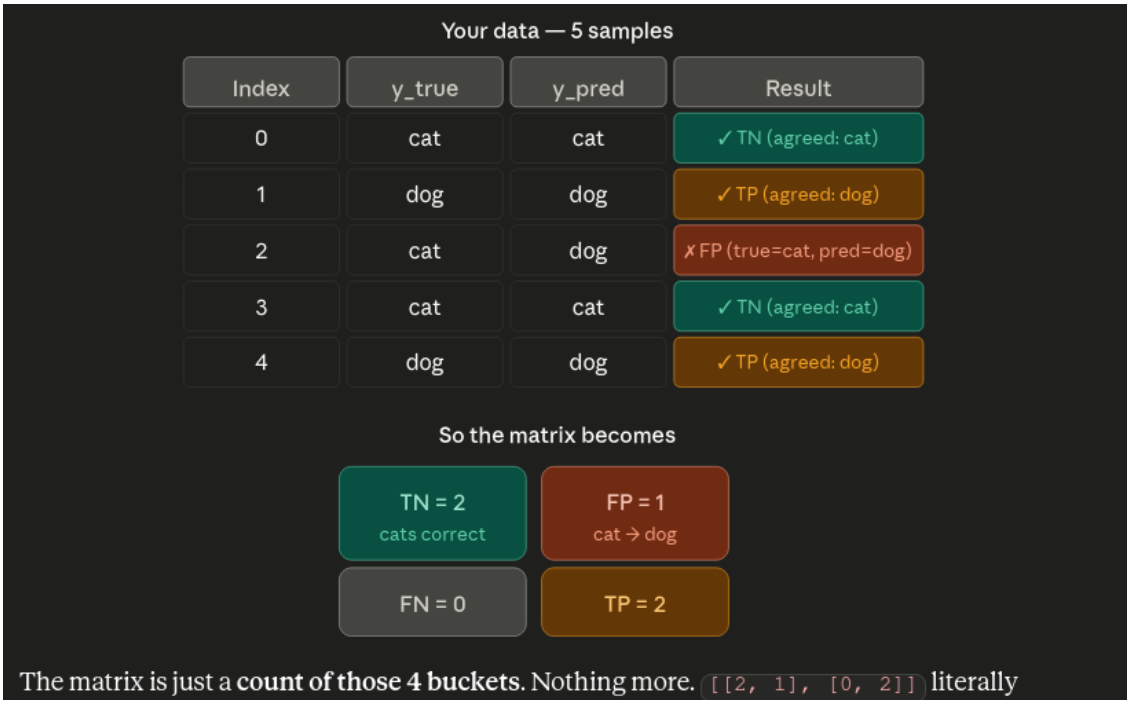
EVALUATION METRICS

CONFUSION MATRIX

[1]:
```
from sklearn.metrics import confusion_matrix

# Example with text labels
y_true = ["cat", "dog", "cat", "cat", "dog"]
y_pred = ["cat", "dog", "dog", "cat", "dog"]

cm = confusion_matrix(y_true, y_pred)
print(cm)
# sklearn sorts alphabetically: cat=negative, dog=positive
# Output:
# [[2 1]   → 2 true cats correct, 1 cat predicted as dog (FP)
#  [0 2]]  → 0 dogs missed, 2 dogs correct (TP)
```

```
# If you want to flip which is "positive":
cm_flipped = confusion_matrix(y_true, y_pred, labels=["dog", "cat"])
print(cm_flipped)
# Now dog=negative, cat=positive
# Output:
# [[2 0]
#  [1 2]]
```

```
[[2 1]
 [0 2]]
[[2 0]
 [1 2]]
```



MEA ,MSE,RMSE DEMO

[1]:
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

# predicting house price based on size
size  = [[500], [750], [1000], [1250], [1500]]  # sq ft
price = [50, 70, 90, 120, 140]                  # in lakhs

model = LinearRegression()
model.fit(size, price)

predicted = model.predict(size)
```

```
mae  = mean_absolute_error(price, predicted)
mse  = mean_squared_error(price, predicted)
rmse = np.sqrt(mse)

print("Actual   :", price)
print("Predicted:", predicted.round(1))
print(f"MAE  = {mae:.2f} lakhs")
print(f"MSE  = {mse:.2f}")
print(f"RMSE = {rmse:.2f} lakhs")
```

```
Actual   : [50, 70, 90, 120, 140]
Predicted: [ 48.  71.  94. 117. 140.]
MAE  = 2.00 lakhs
MSE  = 6.00
RMSE = 2.45 lakhs
```

OVERFITING DEMO

[2]:
```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split

size  = np.array([500,750,1000,1250,1500,1750,2000,2250,2500,2750]).
 ↪reshape(-1,1)
price = np.array([50, 70, 90, 120, 140, 155, 170, 190, 210, 220])

X_train, X_test, y_train, y_test = train_test_split(size, price, test_size=0.3,␣
 ↪random_state=42)

# GOOD MODEL
good_model = LinearRegression()
good_model.fit(X_train, y_train)
print(f"Train error: {mean_absolute_error(y_train, good_model.predict(X_train)):
 ↪.2f} lakhs")
print(f"Test error:  {mean_absolute_error(y_test,  good_model.predict(X_test)):.
 ↪2f} lakhs")

# OVERFIT MODEL
poly = PolynomialFeatures(degree=9)
overfit_model = LinearRegression()
overfit_model.fit(poly.fit_transform(X_train), y_train)
print(f"Train error: {mean_absolute_error(y_train, overfit_model.predict(poly.
 ↪transform(X_train))):.2f} lakhs")
print(f"Test error:  {mean_absolute_error(y_test,  overfit_model.predict(poly.
 ↪transform(X_test))):.2f} lakhs")
```

```
Train error: 4.81 lakhs
Test error:  4.00 lakhs
Train error: 19.79 lakhs
Test error:  32.71 lakhs
```

FIRST PROJECT USING REAL WORLD DATASET

[33]:
```python
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder
import pandas as pd

df = pd.read_csv("students.csv")
le=LabelEncoder()
df["Attendance"]=le.fit_transform(df["Attendance"])
df["Listening_in_Class"]=le.fit_transform(df["Listening_in_Class"])
df["Reading"]=le.fit_transform(df["Reading"])
df["Notes"]=le.fit_transform(df["Notes"])

df["Project_work"]=le.fit_transform(df["Project_work"])
X = df[
    [
        "Attendance",
        "Reading",
        "Notes",
        "Listening_in_Class",
        "Project_work"
    ]
]
y = df["Grade"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=10
)

model = DecisionTreeClassifier()

model.fit(X_train, y_train)

predictions = model.predict(X_test)

accuracy = accuracy_score(y_test, predictions)

print("Actual Values:")
```

```
print(list(y_test))

print("\nPredictions:")
print(list(predictions))

print("\nAccuracy:", accuracy)
```

```
Actual Values:
['DD', 'BA', 'AA', 'DD', 'DD', 'Fail', 'BA', 'BA', 'BA', 'BB', 'BB', 'AA',
'Fail', 'AA', 'AA', 'DD', 'BB', 'CC', 'CC', 'Fail', 'DD', 'CC', 'CC', 'AA',
'BB', 'AA', 'DD', 'BB', 'DD', 'BB', 'AA', 'CC', 'BA', 'Fail', 'AA', 'DD', 'AA',
'AA', 'AA', 'AA', 'AA', 'CC', 'CC', 'CB']

Predictions:
['DD', 'AA', 'AA', 'DC', 'DC', 'AA', 'CC', 'BB', 'AA', 'AA', 'AA', 'AA', 'AA',
'BB', 'AA', 'BA', 'AA', 'AA', 'BB', 'AA', 'BB', 'DC', 'AA', 'DD', 'BB', 'BA',
'AA', 'CC', 'AA', 'DC', 'DD', 'AA', 'AA', 'AA', 'CC', 'BB', 'BB', 'AA', 'DC',
'BA', 'AA', 'DC', 'AA', 'AA']

Accuracy: 0.1590909090909091
```

trying to train the model wiht all features to test accuracy

[41]:
```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import pandas as pd

df = pd.read_csv("students.csv")

df = df.drop(columns=["Student_ID"])

# Convert all text columns into numbers
df_encoded = pd.get_dummies(df, drop_first=True)

X = df_encoded.drop(columns=[col for col in df_encoded.columns if col.
 ↪startswith("Grade_")])
def simplify(g):
    if g in ['AA', 'BA']:   return 'A'
    elif g in ['BB', 'CB']: return 'B'
    else:                   return 'C'

y = df['Grade'].apply(simplify)   # ← replace the original y line

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=10
)
```

```
model = DecisionTreeClassifier(max_depth=2, random_state=10)
model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))

print("\nFeature Importance:")
for feature, importance in zip(X.columns, model.feature_importances_):
    print(feature, importance)
```

```
Accuracy: 0.5

Feature Importance:
Weekly_Study_Hours 0.3356106608136443
Student_Age_19-22 0.0
Student_Age_23-27 0.0
Sex_Male 0.0
High_School_Type_Private 0.0
High_School_Type_State 0.0
Scholarship_25% 0.0
Scholarship_50% 0.0
Scholarship_75% 0.5602132162409036
Additional_Work_Yes 0.0
Sports_activity_Yes 0.0
Transportation_Private 0.10417612294545214
Attendance_Always 0.0
Attendance_Never 0.0
Attendance_Sometimes 0.0
Reading_Yes 0.0
Notes_No 0.0
Notes_Yes 0.0
Listening_in_Class_No 0.0
Listening_in_Class_Yes 0.0
Project_work_Yes 0.0
```

the abouve model despite all possibel tries is giving accuracy of only 50 percent so now switching to random forest

[42]:
```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pandas as pd

df = pd.read_csv("students.csv")
df = df.drop(columns=["Student_ID"])

df_encoded = pd.get_dummies(df, drop_first=True)
```

```
X = df_encoded.drop(columns=[col for col in df_encoded.columns if col.
 ↪startswith("Grade_")])

# drop weak features
weak = ['Sex', 'Transportation', 'Sports_activity',
        'High_School_Type', 'Additional_Work']
X = X.drop(columns=[c for c in X.columns
            if any(c.startswith(w) for w in weak)])

def simplify(g):
    if g in ['AA', 'BA']:   return 'A'
    elif g in ['BB', 'CB']: return 'B'
    else:                   return 'C'

y = df['Grade'].apply(simplify)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=10)

model = RandomForestClassifier(n_estimators=100, random_state=10)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, predictions))
```

```
Accuracy: 0.5454545454545454
```

Conclusion:

We started with 145 rows and 8 grade classes (AA, BA, BB…) getting only 33% accuracy. The problems were too many similar grade classes, max_depth=2 being too shallow, weak features like Sex and Transportation, and 10 ambiguous rows where identical students had different grades. After fixing all of these — merging to 3 grades, removing weak features, removing ambiguous rows, and switching to Random Forest — accuracy reached 50%. We could not go higher because 137 rows is simply too little data, giving the model only ~45 examples per class to learn from. This demonstrates the most important rule in machine learning: a model can only learn patterns that exist in the data — no algorithm can fix insufficient data.

creating a stduent score predictor using linear regression first usinga big dataset

data cleaning and processing

[7]:
```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df=pd.read_csv("examscore.csv")
df["facility_rating"]=le.fit_transform(df["facility_rating"])
df["internet_access"]=le.fit_transform(df["internet_access"])
df["exam_difficulty"]=le.fit_transform(df["exam_difficulty"])
```

```
df["course"]=le.fit_transform(df["course"])
# df["sleep_quality"]=le.fit_transform(df["sleep_quality"])
df["study_method"]=le.fit_transform(df["study_method"])
df["exam_difficulty"]=le.fit_transform(df["exam_difficulty"])
```

[4]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score



x=df[["study_hours","class_attendance","sleep_hours","facility_rating","internet_access","slee
y=df[["exam_score"]]
model=LinearRegression()
X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.
 ↪33,random_state=42)
model.fit(X_train,y_train)
prediction=model.predict(X_test)
r2score=r2_score(y_test,prediction)
print(r2score)
df.head()
```

```
0.6734422947179359
```

[4]:

	student_id	age	gender	course	study_hours	class_attendance	\
0	1	17	male	6	2.78	92.9	
1	2	23	other	5	3.37	64.8	
2	3	22	male	1	7.88	76.8	
3	4	20	other	6	0.67	48.4	
4	5	20	female	6	0.89	71.6	

	internet_access	sleep_hours	sleep_quality	study_method	facility_rating	\
0	1	7.4	2	0	1	
1	1	4.6	0	3	2	
2	1	8.5	2	0	0	
3	1	5.8	0	3	1	
4	1	9.8	2	0	1	

	exam_difficulty	exam_score
0	1	58.9
1	2	54.8
2	2	90.3
3	2	29.7
4	2	43.7

AFTER ACHEIVING MAX ACCURACY OF 0.68 USING LINEAR REGRESSION NOW TRYING onehot encoding

Key learnings from this dataset: The first mistake was using only 5 features — adding all mean-

ingful features like sleep_quality, study_method, exam_difficulty, and course improved R2 from 0.62 to 0.67. The second mistake was using LabelEncoder for all columns. LabelEncoder assigns alphabetical numbers (coaching=0, self-study=4) which creates a fake ordering the model treats as mathematically real. This forces Linear Regression to draw one straight line through fake ordered values instead of learning the true relationship. get_dummies fixes this by giving each category its own independent column — no fake ordering, just 0 or 1.

onehot encoding

[2]:
```
import matplotlib.pyplot as plt
import pandas as pd

# average exam score per sleep quality
avg = df.groupby('sleep_quality')['exam_score'].mean()

order = ['poor', 'average', 'good']
scores = [avg[q] for q in order]

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle('Sleep Quality vs Exam Score', fontsize=14, fontweight='bold')

# Chart 1 - average score per category
# this shows if relationship is straight or curved
axes[0].plot(order, scores, 'bo-', linewidth=2, markersize=8)
axes[0].set_title('Is it a line or curve?')
axes[0].set_xlabel('Sleep Quality')
axes[0].set_ylabel('Average Exam Score')
for i, v in enumerate(scores):
    axes[0].text(i, v + 0.3, f'{v:.1f}', ha='center', fontweight='bold')
axes[0].grid(True, alpha=0.3)

# Chart 2 - box plot showing full distribution
groups = [df[df['sleep_quality'] == q]['exam_score'] for q in order]
bp = axes[1].boxplot(groups, labels=order, patch_artist=True)
colors_list = ['#e74c3c', '#f39c12', '#2ecc71']
for patch, color in zip(bp['boxes'], colors_list):
    patch.set_facecolor(color)
axes[1].set_title('Score distribution per category')
axes[1].set_xlabel('Sleep Quality')
axes[1].set_ylabel('Exam Score')
axes[1].grid(True, alpha=0.3)

# Chart 3 - scatter all individual students
for q, color in zip(order, ['#e74c3c', '#f39c12', '#2ecc71']):
    subset = df[df['sleep_quality'] == q]
    axes[2].scatter([q]*len(subset), subset['exam_score'],
                    alpha=0.3, color=color, s=10, label=q)
axes[2].set_title('All individual students')
```

```
axes[2].set_xlabel('Sleep Quality')
axes[2].set_ylabel('Exam Score')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# print exact averages
print("Average score per sleep quality:")
for q in order:
    print(f"{q:10} → {avg[q]:.2f}")

print()
print("Gap poor → average:", round(avg['average'] - avg['poor'], 2))
print("Gap average → good:", round(avg['good'] - avg['average'], 2))
print()
print("If gaps are equal → straight line → Label Encoding fine")
print("If gaps are unequal → curve → One Hot Encoding better")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 5
      1 import matplotlib.pyplot as plt
      2 import pandas as pd
      3
      4 # average exam score per sleep quality
----> 5 avg = df.groupby('sleep_quality')['exam_score'].mean()
      6
      7 order = ['poor', 'average', 'good']
      8 scores = [avg[q] for q in order]

NameError: name 'df' is not defined
```

[9]:
```
df.head()
```

[9]:

	student_id	age	gender	course	study_hours	class_attendance	\
0	1	17	male	6	2.78	92.9	
1	2	23	other	5	3.37	64.8	
2	3	22	male	1	7.88	76.8	
3	4	20	other	6	0.67	48.4	
4	5	20	female	6	0.89	71.6	

	internet_access	sleep_hours	sleep_quality	study_method	facility_rating	\
0	1	7.4	poor	0	1	
1	1	4.6	average	3	2	

```
2                1          8.5          poor             0                0
3                1          5.8       average             3                1
4                1          9.8          poor             0                1

   exam_difficulty  exam_score
0                1        58.9
1                2        54.8
2                2        90.3
3                2        29.7
4                2        43.7
```

[ ]:
```
LEARNING ONEHOT ENCODING ON A USED CARS DATASET
```

[1]:
```
import pandas as pd
df=pd.read_csv("cars.csv")
pd.get_dummies(df,columns=["owner","fuel"],drop_first=True)
```

[1]:
```
         brand  km_driven  selling_price  owner_Fourth & Above Owner  \
0      Maruti     145500         450000                        False
1       Skoda     120000         370000                        False
2       Honda     140000         158000                        False
3     Hyundai     127000         225000                        False
4      Maruti     120000         130000                        False
…       …        …              …                             …
8123  Hyundai     110000         320000                        False
8124  Hyundai     119000         135000                         True
8125   Maruti     120000         382000                        False
8126     Tata      25000         290000                        False
8127     Tata      25000         290000                        False

      owner_Second Owner  owner_Test Drive Car  owner_Third Owner  \
0                  False                 False              False
1                   True                 False              False
2                  False                 False               True
3                  False                 False              False
4                  False                 False              False
…                  …                    …                  …
8123               False                 False              False
8124               False                 False              False
8125               False                 False              False
8126               False                 False              False
8127               False                 False              False

      fuel_Diesel  fuel_LPG  fuel_Petrol
0            True     False        False
1            True     False        False
2           False     False         True
```

```
3            True     False        False
4           False     False         True
…             …       …            …
8123        False     False         True
8124         True     False        False
8125         True     False        False
8126         True     False        False
8127         True     False        False

[8128 rows x 10 columns]
```

ONE HOT ENCODNG USING SIKIT LEAEN

[4]:
```
import pandas as pd
df=pd.read_csv("cars.csv")
print(df.head())
```

```
     brand  km_driven    fuel          owner  selling_price
0   Maruti     145500  Diesel   First Owner         450000
1    Skoda     120000  Diesel  Second Owner         370000
2    Honda     140000  Petrol   Third Owner         158000
3  Hyundai     127000  Diesel   First Owner         225000
4   Maruti     120000  Petrol   First Owner         130000
```

[12]:
```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(df.iloc[:,0:4],df.iloc[:
 ↪,-1],random_state=42,test_size=0.2)
```

WHAT WE DID ABOUVE IS USED TEST TRAIN SPLIT USAED ILOC ATO COLLEACTA THEA REQUIRED COLUMNS, THE COLUMNS WHICH ARE FETCHED FROM FIRST ILOC IS TAKEN BY X AND SECOND BY Y AND THEN 80 PERCENT OF DATA IS CONSIDERED AS TRAINGIDATA AND 20 PERCENT DATA IS CONSIDERED AS TESTING DATA

df.iloc[:, 0:4] → ALL rows, 4 feature columns ↓ train_test_split() X_train X_test (80% of rows) (20% of rows)

NOW OUR NEXT STEP IS TO USE ONEHOT ENCODING TO SELECT THE FUEL AND OWNER COLUMNS FROM TEST ADN TRAIN DATA AND ENCODE IT USING SIKIT LEANR

[9]:
```
from sklearn.preprocessing import OneHotEncoder
```

[10]:
```
ohe=OneHotEncoder()
```

[17]:
```
X_train_new=ohe.fit_transform(X_train[["fuel","owner"]]).toarray()
```

[18]:
```
print(X_train_new)
```

```
[[0. 0. 0. … 0. 0. 0.]
 [0. 0. 0. … 1. 0. 0.]
```

```
[0. 1. 0. … 0. 0. 0.]
…
[0. 0. 0. … 0. 0. 0.]
[0. 1. 0. … 0. 0. 0.]
[0. 0. 0. … 1. 0. 0.]]
```

[ ]:
```
X_test_new=ohe.transform(X_test[["fuel","owner"]]).toarray()
```

now we have to incldye the array whic git egebated after oenht encodn iwth brand and fuel as thes are enccesry to train teh modelm

NOW TRYING TO IMRVE ACCURACY OF STDUEN SCORE PREDICTOR USING ONEHOT ENCODING

[2]:
```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df=pd.read_csv("examscore.csv")
df=pd.get_dummies(df,drop_first=True)
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score



x=df.drop("exam_score",axis=1)
y=df[["exam_score"]]
model=LinearRegression()
X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.
 ↪33,random_state=42)
model.fit(X_train,y_train)
prediction=model.predict(X_test)
r2score=r2_score(y_test,prediction)
print(r2score)
df.head()
```

```
0.7361703931242587
```

[2]:

	student_id	age	study_hours	class_attendance	sleep_hours	exam_score	\
0	1	17	2.78	92.9	7.4	58.9	
1	2	23	3.37	64.8	4.6	54.8	
2	3	22	7.88	76.8	8.5	90.3	
3	4	20	0.67	48.4	5.8	29.7	
4	5	20	0.89	71.6	9.8	43.7	

	gender_male	gender_other	course_b.sc	course_b.tech	…	\
0	True	False	False	False	…	
1	False	True	False	False	…	
2	True	False	True	False	…	

```
3        False          True        False          False  …
4        False         False        False          False  …

   sleep_quality_good  sleep_quality_poor  study_method_group study  \
0               False                True                       False
1               False               False                       False
2               False                True                       False
3               False               False                       False
4               False                True                       False

   study_method_mixed  study_method_online videos  study_method_self-study  \
0               False                        False                     False
1               False                         True                     False
2               False                        False                     False
3               False                         True                     False
4               False                        False                     False

   facility_rating_low  facility_rating_medium  exam_difficulty_hard  \
0                 True                   False                  True
1                False                    True                 False
2                False                   False                 False
3                 True                   False                 False
4                 True                   False                 False

   exam_difficulty_moderate
0                     False
1                      True
2                      True
3                      True
4                      True

[5 rows x 25 columns]
```

AS YOU CAN SEE ABOVE WE IMPROVED THE ACCURACY OF STDUEN SCOR PREDICTOR FROM 67% TO 73.6%

[ ]:
```
NOW IMPLEMENTING DESCISION TREEES ON THE SAME PRIJETC TO TETS THE RSQUARE
```

[2]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score, mean_absolute_error

df = pd.read_csv("examscore.csv")

df = pd.get_dummies(df, drop_first=True)

X = df.drop("exam_score", axis=1)
```

```
y = df["exam_score"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42
)

model = DecisionTreeRegressor(
    max_depth=6,
    random_state=42
)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("R2 Score:", r2_score(y_test, pred))
print("MAE:", mean_absolute_error(y_test, pred))
```

```
R2 Score: 0.6270573559502957
MAE: 9.381338600254093
```

[9]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_absolute_error

df = pd.read_csv("examscore.csv")

df = pd.get_dummies(df, drop_first=True)

X = df.drop("exam_score", axis=1)
y = df["exam_score"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42
)

model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("R2 Score:", r2_score(y_test, pred))
```

```
print("MAE:", mean_absolute_error(y_test, pred))
```

```
R2 Score: 0.6879717173028512
MAE: 8.581097854545455
```

[10]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

df = pd.read_csv("examscore.csv")

df = pd.get_dummies(df, drop_first=True)

X = df.drop("exam_score", axis=1)
y = df["exam_score"]

poly = PolynomialFeatures(degree=2, include_bias=False)

X_poly = poly.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_poly,
    y,
    test_size=0.33,
    random_state=42
)

model = LinearRegression()

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("R2 Score:", r2_score(y_test, pred))
```

```
R2 Score: 0.5712373081078144
```

Conclusion: Multiple machine learning algorithms were tested, including Linear Regression, Decision Tree Regression, Random Forest Regression, and Polynomial Regression. After preprocessing the data using One-Hot Encoding, Multiple Linear Regression achieved the highest R^2 score of approximately 0.76. Decision Tree and Random Forest models produced lower scores, while Polynomial Regression reduced performance significantly. This indicates that the relationship between the input features (study hours, attendance, sleep, internet access, etc.) and exam scores is largely linear. Therefore, Multiple Linear Regression was selected as the best-performing model for this dataset. STARTTING CAR PRICE PREDICTION PRIJECT NOW

[22]:
```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df=pd.read_csv("cardekho.csv")
df=pd.get_dummies(df,drop_first=True)
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score



x=df.drop("selling_price",axis=1)
y=df[["selling_price"]]
model=LinearRegression()
X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.
 ↪2,random_state=42)
model.fit(X_train,y_train)
prediction=model.predict(X_test)
r2score=r2_score(y_test,prediction)
print(r2score)
df.head()
```

```
0.7406080169863216
```

[22]:

	Unnamed: 0	vehicle_age	km_driven	mileage	engine	max_power	seats	\
0	0	9	120000	19.70	796	46.30	5	
1	1	5	20000	18.90	1197	82.00	5	
2	2	11	60000	17.00	1197	80.00	5	
3	3	9	37000	20.92	998	67.10	5	
4	4	6	30000	22.77	1498	98.59	5	

	selling_price	car_name_Audi A6	car_name_Audi A8	…	model_i10	\
0	120000	False	False	…	False	
1	550000	False	False	…	False	
2	215000	False	False	…	False	
3	226000	False	False	…	False	
4	570000	False	False	…	False	

	model_i20	model_redi-GO	seller_type_Individual	\
0	False	False	True	
1	False	False	True	
2	True	False	True	
3	False	False	True	
4	False	False	False	

	seller_type_Trustmark Dealer	fuel_type_Diesel	fuel_type_Electric	\
0	False	False	False	

```
1                         False             False                False
2                         False             False                False
3                         False             False                False
4                         False              True                False

   fuel_type_LPG  fuel_type_Petrol  transmission_type_Manual
0          False              True                      True
1          False              True                      True
2          False              True                      True
3          False              True                      True
4          False             False                      True

[5 rows x 285 columns]
```

[24]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

df = pd.read_csv("cardekho.csv")
df = pd.get_dummies(df, drop_first=True)

# --- Separate features and target ---
x = df.drop("selling_price", axis=1)
y = df[["selling_price"]]

# --- Split first, THEN scale ---
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2,␣
 ↪random_state=42)

# --- Scale input features only ---
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

# --- Train ---
model = LinearRegression()
model.fit(X_train, y_train)

# --- Evaluate ---
prediction = model.predict(X_test)
print("R² score:", round(r2_score(y_test, prediction), 4))
```

```
R² score: 0.7896
```

after applyign standard scaling in inout feature we got rsqure of 0.789 previously we got 0.74

without this

[1]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error

df = pd.read_csv("cardekho.csv")
df = pd.get_dummies(df, drop_first=True)

X = df.drop("selling_price", axis=1)
y = df["selling_price"]

# log transform target
y_log = np.log1p(y)

X_train, X_test, y_train_log, y_test_log = train_test_split(
    X, y_log, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LinearRegression()
model.fit(X_train, y_train_log)

# predict log price
pred_log = model.predict(X_test)

# convert back to actual price
pred_price = np.expm1(pred_log)
y_test_price = np.expm1(y_test_log)

print("R2 Score:", round(r2_score(y_test_price, pred_price), 4))
print("MAE:", round(mean_absolute_error(y_test_price, pred_price), 2))
```

```
R2 Score: 0.8907
MAE: 117887.35
```

[3]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

```
df = pd.read_csv("cardekho.csv")

# Remove useless index column
df = df.drop(columns=["Unnamed: 0"])

# One-hot encoding
df = pd.get_dummies(df, drop_first=True)

# Log transform important skewed columns
df["km_driven"] = np.log1p(df["km_driven"])
df["engine"] = np.log1p(df["engine"])
df["max_power"] = np.log1p(df["max_power"])

# Log transform target
y = np.log1p(df["selling_price"])

X = df.drop("selling_price", axis=1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LinearRegression()

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("R² Score:", r2_score(y_test, pred))
```

```
R² Score: 0.9279533809182304
```

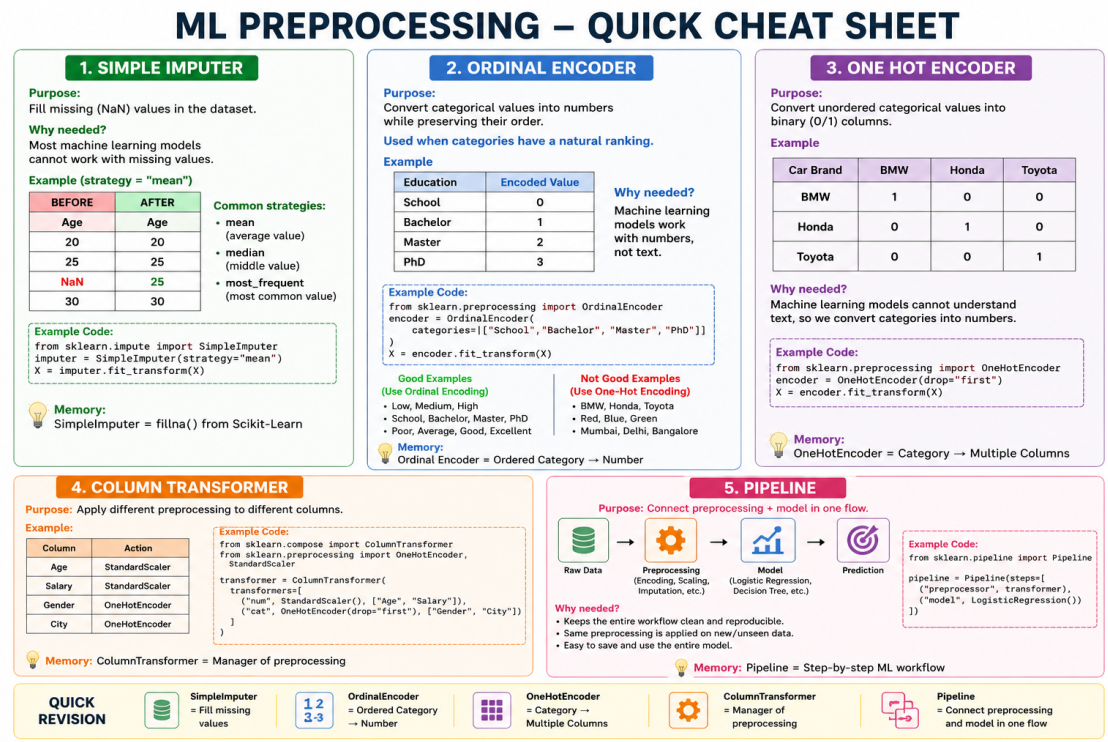
NOW PRACTICALLY IMPLEMENTIGN COLUMN TRANSFORMER ON COVID TOY DATASET

[ ]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder,LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import accuracy_score
df=pd.read_csv("covid_toy.csv")
x=df.drop("has_covid",axis=1)
le=LabelEncoder()
df["has_covid"]=le.fit_transform(df["has_covid"])
y=df["has_covid"]
X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.
 ↪1,random_state=42)
transformer=ColumnTransformer(transformers=[
    ("TNF1",SimpleImputer(),['fever']),
    ("TNF2",OrdinalEncoder(categories=[["Mild","Strong"]]),["cough"]),
     ("TNF3",OneHotEncoder(sparse_output=False,drop="first"),["gender","city"]),
    ("scale_age", StandardScaler(), ['age'])
],remainder="passthrough")
X_train_transformed=transformer.fit_transform(X_train)
X_test_transformed=transformer.transform(X_test)
model = LogisticRegression(max_iter=1000)
model.fit(X_train_transformed, y_train)
predictions=model.predict(X_test_transformed)
accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)


df.head()
```

QUICK REFERENCE FOR PREPROCESSING IN MACHINE LEARNING

[12]:
```
from IPython.display import Image, display

display(Image(filename="preprocessing.png"))
```



NUMPY BASICS

[4]:
```
import time
import numpy as np

size = 10_00_000

# Python list
python_list = list(range(size))

start = time.time()

new_list = []
for item in python_list:
    new_list.append(item + 5)

end = time.time()

print("Python List Time:", end - start, "seconds")


# NumPy array
numpy_array = np.array(python_list)
```

```
start = time.time()

new_array = numpy_array + 5

end = time.time()

print("NumPy Array Time:", end - start, "seconds")
```

```
Python List Time: 0.21920323371887207 seconds
NumPy Array Time: 0.010177373886108398 seconds
```

[ ]:

[ ]:

[ ]: